

Advanced HTTP: HTTPS & Server Events

Section 6, Subtopic 03 — Advanced HTTP

Estimated Duration: 2–3 hours

Theory

Practical

Topics covered: HTTPS setup with self-signed certificates, server events (request, error, close, connection), graceful shutdown

Learning Objectives — This Subtopic

- Create an HTTPS server using self-signed TLS certificates in Node.js
- Listen for and respond to core server events: `request`, `connection`, `error`, and `close`
- Implement a graceful shutdown pattern using process signals and `server.close()`

1. HTTPS Setup Basics

Every HTTP server built so far transmits data in plain text. HTTPS wraps that same HTTP traffic inside a TLS (Transport Layer Security) encryption layer, preventing eavesdropping and tampering. Node.js ships with a built-in `https` module that mirrors the `http` API almost exactly — the only difference is that it requires a TLS certificate and private key.

1.1 Generating a Self-Signed Certificate

Production servers use certificates issued by a trusted Certificate Authority (CA). For local development, a self-signed certificate is sufficient. OpenSSL generates both the private key and the certificate in one command.

Run this in your project directory:

```
openssl req -x509 -newkey rsa:2048 -keyout key.pem -out cert.pem -days 365 -nodes -subj "/CN=localhost"
```

Output:

```
Generating a 2048 bit RSA private key
.....++
.....++
writing new private key to 'key.pem'
-----
```

This creates two files in your working directory:

File	Purpose
key.pem	The private key — never share or commit this to version control
cert.pem	The self-signed certificate — presented to clients during the TLS handshake

Note: On Windows without OpenSSL installed, you can use Git Bash (which bundles OpenSSL) or install OpenSSL via `winget install ShiningLight.OpenSSL`. The command above works identically on all platforms.

Warning: Never commit `key.pem` to a Git repository. Add it to your `.gitignore` immediately. Leaking a private key compromises all encrypted traffic.

1.2 Creating an HTTPS Server

Stage 1 A minimal HTTPS server that reads the certificate files and responds to every request:

```
https-server.js
```

```
const https = require('https');
const fs = require('fs');

const options = {
  key: fs.readFileSync('key.pem'),
  cert: fs.readFileSync('cert.pem')
};

const server = https.createServer(options, (req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Secure connection established\n');
});

server.listen(3443, () => {
  console.log('HTTPS server running at https://localhost:3443/');
});
```

Run the server:

```
node https-server.js
```

Output:

```
HTTPS server running at https://localhost:3443/
```

Open `https://localhost:3443/` in a browser. Because the certificate is self-signed, the browser displays a security warning. This is expected in development — click through to proceed. Alternatively, test with `curl` :

```
curl -k https://localhost:3443/
```

Output:

```
Secure connection established
```

The `-k` flag tells `curl` to accept self-signed certificates.

What is happening?

- `fs.readFileSync` loads the key and certificate into memory as buffers. These are read synchronously at startup, which is acceptable since the server has not begun accepting connections yet.
- The `options` object is passed as the first argument to `https.createServer`. Under the hood, Node.js creates a TLS-wrapped TCP socket.
- The request handler callback is identical to what you would write for `http.createServer` — the encryption is transparent to your application code.

Analogy: Think of HTTP as sending a postcard — anyone handling it along the way can read the message. HTTPS is like putting that postcard inside a sealed, tamper-evident envelope. The message (HTTP request/response) is the same; the envelope (TLS) protects it in transit.

1.3 HTTP and HTTPS Side by Side

Stage 2 It is common in development to run both an HTTP and HTTPS server simultaneously. This lets you test redirect behaviour:

dual-server.js

```
const http = require('http');
const https = require('https');
const fs = require('fs');

const tlsOptions = {
  key: fs.readFileSync('key.pem'),
  cert: fs.readFileSync('cert.pem')
};

// HTTP server – redirects to HTTPS
http.createServer((req, res) => {
  const redirectUrl = `https://localhost:3443${req.url}`;
  res.writeHead(301, { Location: redirectUrl });
  res.end();
}).listen(3080, () => {
  console.log('HTTP server on http://localhost:3080/ (redirects to HTTPS)');
});

// HTTPS server – handles actual requests
https.createServer(tlsOptions, (req, res) => {
```

```
res.writeHead(200, { 'Content-Type': 'text/plain' });
res.end(`You arrived securely at ${req.url}\n`);
}).listen(3443, () => {
  console.log('HTTPS server on https://localhost:3443/');
});
```

```
node dual-server.js
```

Output:

```
HTTP  server on http://localhost:3080/ (redirects to HTTPS)
HTTPS server on https://localhost:3443/
```

Test the redirect:

```
curl -kL http://localhost:3080/hello
```

Output:

```
You arrived securely at /hello
```

The `-L` flag instructs `curl` to follow redirects. The HTTP server issues a `301 Moved Permanently` response, and the client follows it to the HTTPS endpoint.

2. Server Events

The HTTP server object returned by `http.createServer()` (and `https.createServer()`) is an `EventEmitter`. The request handler callback you have been passing is actually a shorthand — it attaches a listener for the `request` event. The server emits several other events that give you fine-grained control over the server lifecycle.

2.1 The `request` Event

These two patterns are equivalent:

request-event.js

```
const http = require('http');

// Pattern A: callback shorthand (what you have been using)
const serverA = http.createServer((req, res) => {
  res.end('Pattern A\n');
});

// Pattern B: explicit event listener
const serverB = http.createServer();
serverB.on('request', (req, res) => {
  res.end('Pattern B\n');
});

serverA.listen(3001, () => console.log('Server A on :3001'));
serverB.listen(3002, () => console.log('Server B on :3002'));
```

node request-event.js

Output:

```
Server A on :3001
Server B on :3002
```

Both servers behave identically. Pattern B is useful when you want to attach multiple listeners or configure the server before binding the request handler.

2.2 The `connection` Event

The `connection` event fires every time a new TCP socket is established — before any HTTP data is exchanged. This is useful for logging or tracking active connections.

connection-event.js

```
const http = require('http');

let connectionCount = 0;

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end(`Total connections so far: ${connectionCount}\n`);
});
```

```
});  
  
server.on('connection', (socket) => {  
  connectionCount++;  
  const remote = `${socket.remoteAddress}:${socket.remotePort}`;  
  console.log(`[connection #${connectionCount}] New TCP connection from ${remote}`);  
});  
  
server.listen(3000, () => {  
  console.log('Server listening on http://localhost:3000/');  
});
```

```
node connection-event.js
```

Output:

```
Server listening on http://localhost:3000/
```

In another terminal, make two requests:

```
curl http://localhost:3000/
```

```
curl http://localhost:3000/
```

Output:

```
[connection #1] New TCP connection from ::1:54321  
[connection #2] New TCP connection from ::1:54322
```

What is happening? Each `curl` call opens a fresh TCP connection. The `connection` event fires with the raw `net.Socket` object, which exposes the client's IP address and port. Browsers may reuse connections via HTTP keep-alive, so you may see fewer `connection` events than `request` events.

Key Insight: The `connection` event gives you access to the raw TCP socket. The `request` event gives you the parsed HTTP request and response objects. They fire at different layers of the network stack: TCP vs HTTP.

2.3 The `error` Event

If the server encounters an error (most commonly, the port is already in use), it emits an `error` event. Without a listener, the process crashes with an unhandled exception.

`error-event.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('OK\n');
});

server.on('error', (err) => {
  if (err.code === 'EADDRINUSE') {
    console.error(`Port ${err.port} is already in use. Choose a different port.`);
  } else {
    console.error('Server error:', err.message);
  }
  process.exit(1);
});

server.listen(3000, () => {
  console.log('Server listening on http://localhost:3000/');
});
```

To test this, start the server in one terminal, then run the same script in a second terminal:

```
node error-event.js
```

Output:

```
Server listening on http://localhost:3000/
```

In a second terminal:

```
node error-event.js
```

Output:

```
Port 3000 is already in use. Choose a different port.
```


Without the `error` listener, Node.js would throw an unhandled `Error: listen EADDRINUSE` and crash with a stack trace. Handling the event gracefully gives you the opportunity to log a clear message, retry on a different port, or shut down cleanly.

2.4 The `close` Event

The `close` event fires when the server stops accepting new connections, after `server.close()` is called and all existing connections have ended.

`close-event.js`

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.end('Shutting down soon...\n');
});

server.on('close', () => {
  console.log('Server has fully shut down.');
```

`node close-event.js`

Output:

```
Server listening on http://localhost:3000/
Will shut down in 5 seconds...
Calling server.close()...
Server has fully shut down.
```

The `close` event is the right place to perform cleanup: closing database connections, flushing logs, or notifying monitoring systems. Note that `server.close()` does not forcefully terminate existing connections — it waits for them to finish naturally.

2.5 Graceful Shutdown Pattern

Stage 3 A production-ready pattern combines signal handling with the `close` event to shut down cleanly when the process receives `SIGINT` (Ctrl+C) or `SIGTERM`:

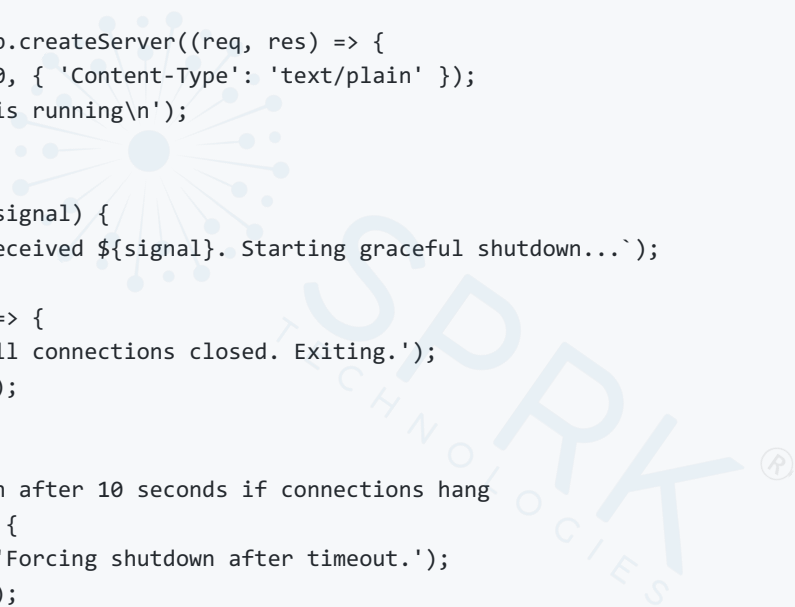
graceful-shutdown.js

```
const http = require('http');

const server = http.createServer((req, res) => {
  res.writeHead(200, { 'Content-Type': 'text/plain' });
  res.end('Server is running\n');
});

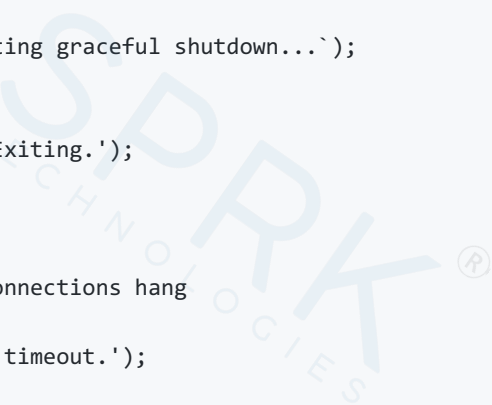
function shutdown(signal) {
  console.log(`\nReceived ${signal}. Starting graceful shutdown...`);

  server.close(() => {
    console.log('All connections closed. Exiting.');
```



```
    process.exit(0);
  });

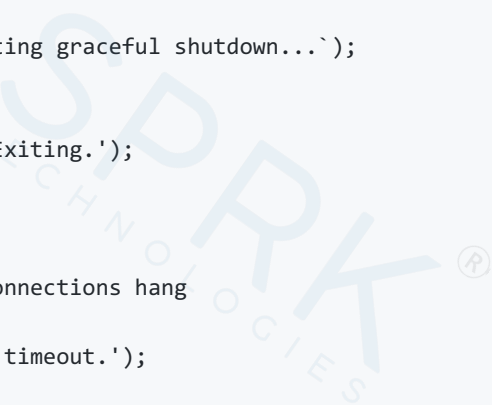
  // Force shutdown after 10 seconds if connections hang
  setTimeout(() => {
    console.error('Forcing shutdown after timeout.');
```



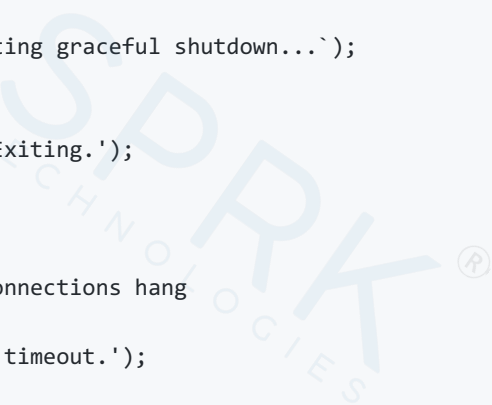
```
    process.exit(1);
  }, 10000);
}

process.on('SIGINT', () => shutdown('SIGINT'));
process.on('SIGTERM', () => shutdown('SIGTERM'));

server.listen(3000, () => {
  console.log('Server listening on http://localhost:3000/');
```



```
  console.log('Press Ctrl+C to trigger graceful shutdown');
```



```
});
```

`node graceful-shutdown.js`

Output:

```
Server listening on http://localhost:3000/
Press Ctrl+C to trigger graceful shutdown
^C
Received SIGINT. Starting graceful shutdown...
All connections closed. Exiting.
```

Note: Windows does not support `SIGTERM` natively. On Windows, `SIGINT` fires on Ctrl+C and is the primary signal to handle. In production, process managers like PM2 (covered in Section 14) handle cross-platform signal management.

Cross-reference: URL parsing (the legacy `url` module, the WHATWG `URL` class, `URLSearchParams`, and the `querystring` module) is covered in detail in **06-02 Server Creation**, Sections 2–3. That subtopic also includes a comparison table, code examples for all APIs, and a combined server example demonstrating URL parsing with query-based filtering.

Summary

Concept	Key Point
HTTPS	Uses <code>https.createServer(options, handler)</code> with TLS key and certificate; identical API to HTTP otherwise
Self-signed certificates	Generated with <code>openssl req -x509</code> ; suitable for development only; browsers show a warning
HTTP → HTTPS redirect	

Concept	Key Point
	Run both servers; HTTP responds with <code>301</code> and a <code>Location</code> header pointing to HTTPS
<code>request</code> event	Fires for every incoming HTTP request; passing a callback to <code>createServer()</code> is shorthand for <code>.on('request', ...)</code>
<code>connection</code> event	Fires when a new TCP socket is established; provides the raw <code>net.Socket</code> object
<code>error</code> event	Fires on server errors (e.g., <code>EADDRINUSE</code>); must be handled to prevent unhandled exceptions
<code>close</code> event	Fires after <code>server.close()</code> once all connections end; used for cleanup
Graceful shutdown	Listen for <code>SIGINT</code> / <code>SIGTERM</code> , call <code>server.close()</code> , set a force-exit timeout
URL parsing & <code>URLSearchParams</code>	Covered in 06-02 Server Creation , Sections 2–3 (legacy <code>url.parse()</code> , WHATWG <code>URL</code> class, <code>URLSearchParams</code>)

Interview Questions

1. What is the primary difference between HTTP and HTTPS in Node.js?

Expected answer: HTTPS wraps HTTP inside a TLS encryption layer. In Node.js,

`https.createServer()` requires a TLS options object containing at minimum a private key and certificate. The request handler API is identical to HTTP.

2. Why does the browser show a security warning for self-signed certificates?

Expected answer: The browser cannot verify the certificate's authenticity because it was not issued by a trusted Certificate Authority. The certificate is cryptographically valid, but its issuer is not in the browser's trust store.

3. Explain the difference between the `connection` and `request` events on an HTTP server.

Expected answer: `connection` fires at the TCP level when a new socket is established. `request` fires at the HTTP level when a complete HTTP request is received on that socket. A single TCP connection can carry multiple HTTP requests (via keep-alive), so there may be more `request` events than `connection` events.

4. What happens if you do not attach a listener for the `error` event on an HTTP server?

Expected answer: Node.js throws an unhandled exception and the process crashes. Attaching an `error` listener allows you to handle the error gracefully — log it, retry, or exit cleanly.

5. How would you implement a graceful shutdown for a Node.js HTTP server?

Expected answer: Listen for process signals (`SIGINT` , `SIGTERM`), call `server.close()` to stop accepting new connections, wait for existing connections to finish, then exit. A timeout should force-exit the process if connections do not close within a reasonable period.

6. What is the difference between `URLSearchParams.get()` and `URLSearchParams.getAll()` ?

Expected answer: `get()` returns the first value for the given key (or `null` if absent). `getAll()` returns an array of all values for that key, which is necessary when a parameter appears multiple times in the URL.

Cross-reference: Additional interview questions on URL parsing (`url.parse()` vs `new URL()` , query string handling, and `URLSearchParams`) can be found in **06-02 Server Creation**.

Practical Assignment: Build a Filtered Book API with HTTPS and Graceful Shutdown

Objective: Build an HTTPS server that serves a collection of books, supports filtering and sorting via query parameters, logs server events, and shuts down gracefully.

1. Generate a self-signed certificate and private key using `openssl` in your project directory.
2. Create a file called `book-server.js` with an array of at least 8 book objects. Each book should have: `id`, `title`, `author`, `genre`, `year`, and `price`.
3. Create an HTTPS server that listens on port `3443`.
4. Implement a `/books` endpoint that:
 - Returns all books as JSON when no query parameters are provided
 - Filters by `genre` when `?genre=fiction` is supplied
 - Filters by `author` when `?author=...` is supplied (case-insensitive)
 - Sorts by any valid field when `?sort=year` or `?sort=price` is supplied
 - Supports combining filters: `?genre=fiction&sort=price`
5. Return a `404` JSON response for any path other than `/books`.
6. Attach listeners for the `connection`, `error`, and `close` events. Log a message for each.
7. Implement graceful shutdown: listen for `SIGINT`, call `server.close()`, and force-exit after 5 seconds.
8. Test all endpoints using `curl -k` and verify the correct JSON responses.

Expected Output

Output:

```
$ node book-server.js
HTTPS Book Server running at https://localhost:3443/books
Press Ctrl+C to shut down gracefully

$ curl -k https://localhost:3443/books?genre=fiction&sort=year
[
  { "id": 3, "title": "1984", "author": "George Orwell", "genre": "fiction", "year": 1949, "price":
  9.99 },
  { "id": 1, "title": "Dune", "author": "Frank Herbert", "genre": "fiction", "year": 1965, "price":
  12.99 },
  ...
]
```

```
] 
```

```
$ # Press Ctrl+C in the server terminal  
Received SIGINT. Starting graceful shutdown...  
All connections closed. Server shut down.
```

Bonus Challenge

Add pagination support: accept `?page=1&limit=3` query parameters. Return only the slice of results for that page, and include a `X-Total-Count` header with the total number of matching results before pagination.

